[GPU 똑똑하게 쓰기 (1)] 모델 학습은 더 쉽게, GPU 활용은 더 효율적으로 - Managed Training과 Mixed Workload

클라우드
2026. 9. 30. 16:00
https://blog.naver.com/post6155/224418808063



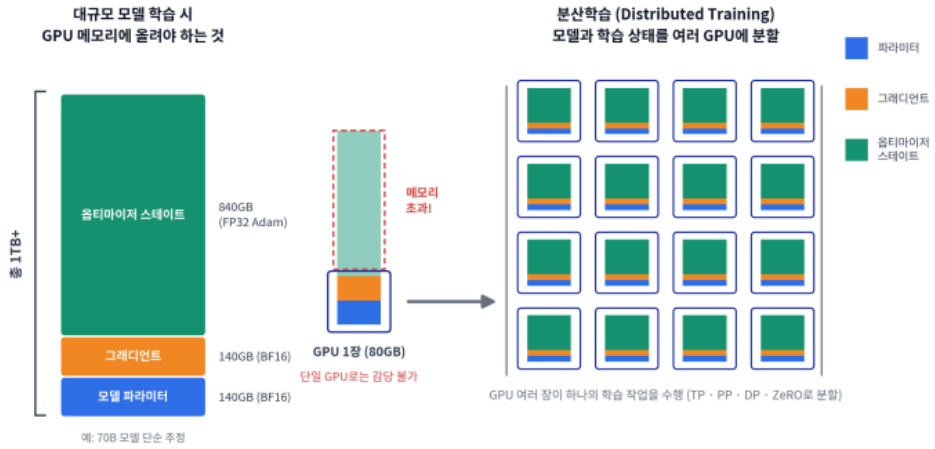
> 모델이 커지면서 분산학습은 필수가 되었지만, 인프라·분산 소프트웨어·운영에 걸친 준비 부담과 항상 부족한 GPU 공급이 학습의 시작을 가로막습니다. Managed Training 서비스로 진입 장벽을 낮추고, 서빙 GPU의 유휴 시간을 활용하는 Mixed Workload 기술을 살펴봅니다.

핵심 인사이트

- 분산학습의 진입 장벽은 학습 코드뿐 아니라 인프라(GPU 클러스터·고속 네트워크·스토리지), 분산 소프트웨어(병렬화 전략·버전 정합), 운영(스케줄링·모니터링·장애 대응)의 세 계층을 함께 준비해야 한다는 데 있습니다. **Managed Training 서비스는 이 부담을 줄여 학습 시작까지의 준비 기간을 단축합니다.**
- 서빙 GPU는 피크 기준으로 준비되므로 심야·휴일에 유휴 시간이 반복될 수 있습니다. **Mixed Workload는 이 유휴 GPU를 학습에 대여하고, 서빙에 다시 필요할 때 회수(Loaning & Reclaim)해 보유 GPU의 활용도를 높입니다.**
- 회수 가능한 GPU를 학습에 활용하려면 중단에 따른 학습 손실을 줄이고, GPU 구성이 달라져도 학습을 재개할 수 있어야 합니다. **Concurrent Checkpointing은 체크포인트 저장과 학습을 겹쳐 진행분 손실을 줄이고, Checkpoint Resharding은 변경된 GPU 구성에서도 학습을 이어갈 수 있게 합니다.**

좋은 모델을 확보해도 학습은 끝나지 않는다

지난 리포트에서는 LLM 서빙, 즉 확보한 모델을 낮은 지연시간과 낮은 비용으로 서비스하는 기술을 살펴봤습니다. 하지만 모델을 서비스에 올렸다고 해서 모델의 학습이 끝나는 것은 아닙니다. 도메인 데이터는 계속 갱신되고, 사용자 피드백을 반영한 튜닝도 반복됩니다. 새로운 오픈웨이트(Open-weight) 모델이 등장할 때마다 원하는 데이터로 추가 학습하여 성능을 올리려는 수요도 꾸준합니다. 특히 대규모 자연어 모델을 전체 학습하거나 포스트 트레이닝(Post Training)하려면 모델 파라미터(Parameter)뿐 아니라 그래디언트(Gradient)와 옵티마이저 스테이트(Optimizer State)까지 GPU 메모리에 올려야 합니다. 모델 규모가 커질수록 단일 GPU 메모리와 연산 성능으로는 이를 감당하기 어려워지고, 다수의 GPU를 묶어 하나의 학습 작업을 수행하는 분산학습(Distributed Training)이 필요해집니다.



[그림1] 대규모 모델 학습에서 분산학습이 필요한 이유 (출처: 삼성SDS)

문제는 이러한 분산학습에서는 GPU 간 통신, 데이터 입출력, 병렬화 방식, 장애 대응에 따라 GPU가 실제로 연산을 수행하지 못하고 대기하고 있는 시간이 크게 늘어날 수 있다는 점입니다. GPU 가격이 높아질수록 이러한 대기 시간은 비용 손실로 직결됩니다. 따라서 대규모 모델 학습에서 중요한 것은 단순히 GPU 개수를 많이 확보하는 것이 아니라, "확보된 GPU 시간을 얼마나 최대한 실제 학습 연산에 사용할 수 있느냐"입니다. 이 글에서는 분산학습 환경의 구축과 운영부터 이미 보유한 GPU 유휴 시간 활용, 학습 중단에 따른 GPU 낭비를 줄이는 방법까지, GPU 활용률을 높이기 위해 고려해야 할 주요 요소들에 대해 살펴보겠습니다.

왜 GPU를 늘려도 학습은 빨라지지 않을까?

- **모델 개발자는 복잡한 분산학습 환경에서 자유롭고 싶다**

모델 개발자가 집중하고 싶은 것은 모델 아키텍처와 데이터, 학습 코드입니다. 하지만 실제로 여러 GPU를 이용해 학습을 시작하려면 GPU가 있는 노드의 인프라, 분산 프레임워크, 운영이라는 3개의 계층이 함께 유기적으로 준비되어야 합니다. 그리고 이 세 계층을 어떻게 구성하는지에 따라 같은 종류와 개수의 GPU를 사용하더라도 실제 학습에 쓰이는 GPU 시간은 크게 달라질 수 있습니다.

첫 번째는 인프라 계층입니다. 분산학습은 GPU 서버 여러 대를 연결하는 것만으로 끝나지 않습니다. 학습에 참여하는 여러 GPU가 학습 스텝마다 그래디언트를 모으고 나누는 집합 통신(예: All-Reduce)을 반복합니다. 모델 규모와 GPU 수가 커질수록 GPU 사이에 이동하는 데이터도 증가합니다. 이때 노드 간 네트워크 대역폭이 충분하지 않으면 GPU는 연산을 끝내고도 다른 GPU와의 통신이 끝날 때까지 대기해야 합니다. GPU의 연산 성능은 충분히 고속이지만 네트워크가 이를 따라가지 못하는 경우에 고가의 GPU가 장기 대기하는 상황이 발생하는 것입니다. 이러한 이유로 대규모 분산학습에서 Infiniband나 RDMA와 같은 고속 네트워크가 중요합니다. 스토리지 역시 GPU 활용률에 직접적인 영향을 줍니다. 다수의 학습 프로세스가 동시에 학습 데이터를 읽는 동안 데이터 입출력이 병목이 되거나, 대규모 모델 중간 학습 가중치인 체크포인트(Checkpoint)를 저장하는 동안 학습이 지연된다면, 해당 지연 시간 역시 GPU를 활용하지 못하는 시간이 됩니다. 따라서, 분산학습 환경에서는 GPU뿐 아니라 네트워크와 스토리지 등의 인프라까지 고려해야 합니다.

두 번째는 분산 소프트웨어 계층입니다. 대규모 모델은 한 장의 GPU에 모두 올릴 수 없기 때문에 모델 파라미터 등의 데이터를 여러 GPU에 어떻게 나눌 것인지 설정해야 합니다. 같은 모델을 여러 GPU에 복제하고 학습 데이터를 나눠 처리하는 데이터 병렬(Data Parallelism; DP), 하나의 모델 텐서(Tensor)를 여러 GPU에 분할

하는 텐서 병렬(Tensor Parallelism; TP), 모델의 레이어를 여러 스테이지로 나누는 파이프라인 병렬(Pipeline Parallelism; PP), 그리고 파라미터, 옵티마이저 스테이트, 그래디언트 등을 여러 GPU에 분산하여 GPU 메모리 사용량을 줄이는 ZeRO와 같은 다양한 방식이 사용됩니다. 각 방식은 필요한 GPU 메모리와 통신량이 서로 다릅니다. 따라서 모델 크기와 GPU 수, 노드 구성, 네트워크 환경에 따라 여러 병렬화 방식을 보통 조합합니다. 잘 맞는 조합을 선택하면 GPU를 효율적으로 사용할 수 있지만, 반대로 통신량이 지나치게 많거나 GPU 연산 부하가 균일하지 않을 시에는 GPU 개수를 늘리고도 기대한 만큼의 학습 속도가 나오지 않을 수도 있습니다. 여기서 NCCL과 같은 GPU 통신 라이브러리, CUDA와 GPU 드라이버, 학습 프레임워크 간 버전 정합성, 컨테이너 환경, 쿠버네티스 환경까지 맞춰야 합니다.

세 번째는 운영 계층입니다. 분산학습 환경을 구축하였다고 해도 운영에서도 중요 요소가 남아 있습니다. GPU는 여러 사용자와 프로젝트가 사용하는 "제한"된 자원입니다. 별도의 관리 체계 없이 여러 사용자가 GPU 클러스터를 공유하면 일부 사용자가 GPU를 장시간 점유하여 다른 사용자는 GPU를 사용할 수 없는 상황이 생길 수 있습니다. 따라서 실제 운영 환경에서는 학습 Job(여러 GPU가 배정되는 학습 작업이 수행되는 단위)의 대기열과 우선순위를 관리하는 스케줄링, 사용자와 프로젝트 간 자원 할당 및 격리, GPU 상태와 학습 로그 모니터링, 체크포인트 관리, 장애 감지와 학습 재시작까지 여러 운영적 요소가 필요합니다. 이러한 운영 체계는 편의 기능을 넘어서 GPU 클러스터의 규모가 커질수록 GPU를 실제 학습에 사용할 수 있는 시간을 좌우할 수 있는 요소입니다.



[그림2] 분산학습을 시작하기 위해 준비해야 하는 세 개의 계층 (출처: 삼성SDS)

- **대규모 학습에서는 장애 대응도 인프라의 일부이다**

분산학습의 규모가 커질수록 운영이 중요한 또 하나의 이유는 장애입니다. 하나의 학습 Job이 사용하는 GPU, 네트워크 장비 등이 많아질수록 그중 일부에서 문제가 발생할 확률도 높아집니다. Meta가 2024년 7월에 공개한 Llama3 논문에 따르면, 16,384장의 H100 GPU를 이용해 405B 모델을 54일간 학습하는 동안 예기치 못한 중단이 총 419건 발생하였습니다. 이를 평균적으로 따진다면 약 3시간에 한 번꼴입니다. 원인 가운데 GPU 자체 문제가 148회, GPU 메모리(HBM3) 관련 문제가 72회였으며, 네트워크 스위치와 케이블 관련 문제도 35회 발생했습니다.[1]

문제는 분산학습에서는 수천 개의 GPU 중 하나라도 장애가 나는 경우에도 전체 Job이 중단될 수 있습니다. GPU 수가 늘어날수록 장애를 완전히 피하는 것보다, 장애가 발생한다는 전제하에 얼마나 빠르게 감지하고 복구하는지가 더 중요해지는 이유입니다. 실제로 Meta는 자동화된 장애 감지와 복구 체계를 운영하면서 유효 학습 시간(Effective Training Time)을 90% 이상으로 유지했다고 밝혔습니다. 대규모 학습에서 장애 대응

은 문제가 발생했을 때 사람이 처리하는 사후 작업이 아니라, 학습 환경 자체가 갖춰야 할 운영 기능에 가까워짐을 시사합니다.

따라서 분산학습 환경에는 GPU와 학습 상태를 지속적으로 모니터링하고, 문제가 발생한 경우 학습 Job을 신속하게 복구할 수 있는 체계가 필요합니다. 다만 학습을 재실행할 수 있다는 것과 장애로 인한 중단 직전까지의 학습 정보를 온전히 지킬 수 있다는 것은 별개의 문제입니다. 중단 시 얼마만큼의 학습 결과가 유실되는지는 체크포인트의 주기에 달려있으며, 이는 뒤에서 다시 살펴보겠습니다.

- **그렇다면 이런 분산학습 환경을 직접 구축해야 할까?**

결국 분산학습에서 필요한 것은 단순한 GPU 클러스터가 아닙니다. 고속 네트워크와 스토리지를 포함한 인프라 환경, 분산학습 실행을 위한 프레임워크 환경, 그리고 장애와 정책 등 관련된 각종 운영 체계를 함께 갖춰야 합니다. 이를 직접 구축할지에 대한 판단을 내리기 위해서 몇 가지 질문을 먼저 확인할 필요가 있습니다:

- 인프라, 분산 프레임워크, 플랫폼 운영을 각각 담당할 전문 인력이 존재하는가?
- 대규모 학습 수요가 지속적으로 발생해 전용 클러스터와 운영 조직을 유지할 만큼 충분한가?
- 장애를 자동으로 감지하고 학습 Job을 복구할 수 있는 체계를 갖추고 있는가?
- 여러 조직과 사용자가 GPU를 공유할 때 효율적으로 운영할 정책과 도구가 있는가?

대규모 학습이 핵심 사업이고 학습 수요가 지속적으로 발생한다면 이러한 환경을 직접 구축하는 것이 장기적으로는 유리할 수 있습니다. 반면 학습 수요가 프로젝트 단위로 발생하거나, 별도의 플랫폼 운영 조직을 유지하기 어려운 환경에서는 분산학습 자체보다 이를 위한 인프라와 운영 환경을 준비하는 데 더 많은 시간과 비용이 들어갈 수 있습니다. 이때 Managed Training 서비스가 하나의 대안이 됩니다.

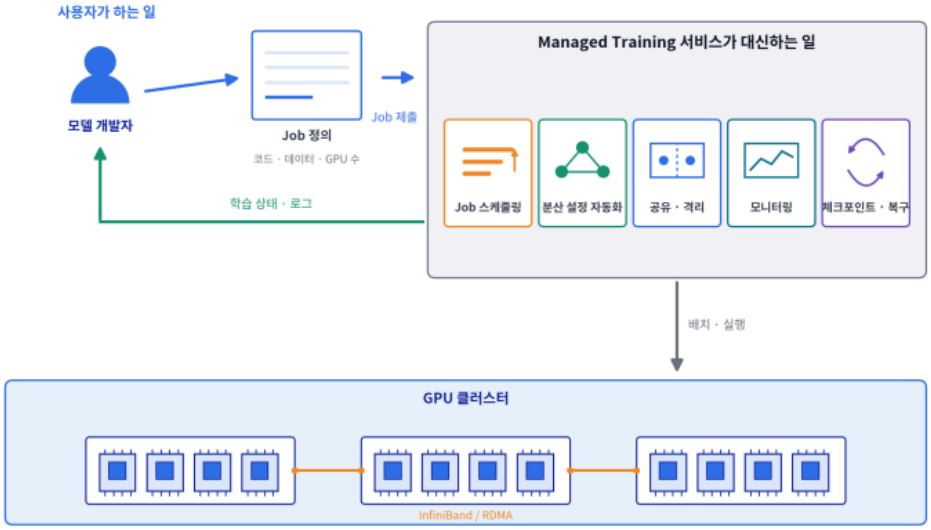
- **해법: 분산학습의 인프라와 운영 부담을 Managed Training 서비스로 넘긴다**

Managed Training 서비스를 적용하게 되면 앞서 말했던 인프라, 분산 소프트웨어 및 운영 영역을 플랫폼으로 옮기게 됩니다.

이를 통해 모델 개발자는 모델과 데이터, 학습 코드 등 학습 결과와 직접 관련된 영역에만 집중할 수 있게 됩니다. 반면 플랫폼은 GPU 노드의 할당과 분산 실행 환경 구성, 학습 상태 모니터링, 장애 발생 시 Job 복구 등 공통적인 운영 기능을 담당합니다. 이를 통해 얻을 수 있는 효과는 크게 세 가지입니다:

- 분산학습 환경을 준비하는 시간을 줄일 수 있습니다. 사용자가 매번 GPU 노드와 네트워크, 컨테이너 환경을 직접 구성하는 대신 학습에 필요한 정의를 통해 학습 Job을 실행할 수 있습니다.
- GPU 클러스터의 활용률을 높일 수 있습니다. 여러 사용자의 학습 Job을 하나의 GPU Pool에서 관리하고 공평하게 배분하면, 각 사용자가 GPU를 각각 확보해 두는 것보다 사용자들이 속한 조직 전체의 유휴 자원을 줄일 수 있습니다.
- 장애 대응으로 손실되는 GPU 시간을 줄일 수 있습니다. GPU와 학습 상태를 지속적으로 모니터링하고 문제가 발생한 Job을 빠르게 복구할 수 있다면, 사람이 장애를 발견하고 대응할 때까지 낭비되는 시간을 줄일 수 있습니다.

Samsung Cloud Platform의 Simple AI Training이 Managed Training 서비스의 한 사례로 이러한 방향으로 분산학습 환경을 제공합니다. 모델 개발자가 인프라를 매번 직접 구성하지 않고 Job 단위로 학습을 실행할 수 있도록 하며, GPU 클러스터의 공유와 자원 관리, GPU 상태 모니터링과 학습 로그 조회 등 분산학습 운영에 필요한 기능을 플랫폼 차원에서 제공합니다. 결국 Managed Training 서비스가 해결하려는 문제는 단순히 여러 GPU를 사용하여 모델 학습 Job을 실행하는 것이 아닙니다. 비싼 GPU가 환경 구성이나 자원 대기, 장애 대응 때문에 멈춰 있는 시간을 줄이고, 확보한 GPU 시간을 실제 학습 연산에 더 많이 사용할 수 있도록 만드는 것이 핵심입니다.



[그림3] Managed Training 서비스의 역할 분담 (출처: 삼성SDS)

하지만 분산학습 환경을 효율적으로 운영한다고 해서 모든 GPU 활용 문제가 해결되는 것은 아닙니다. 서비스 트래픽에 맞춰 확보한 GPU가 특정 시간대에는 활용되고 있지 않을 수 있고, 학습이 중단될 때 진행 분이 유실되거나 학습 중간값을 저장하는 과정에서 GPU가 대기하는 또 다른 형태의 비효율 문제도 발생할 수 있습니다. 결국 다음 질문은 이러한 문제에서 자연스럽게 이어집니다. "이미 확보한 GPU를 어떻게 더 많이 실제 연산에 사용할 것인가?"

학습 GPU는 부족한데, 서빙 GPU를 활용할 수 없을까?

- **GPU 부족은 총량만의 문제가 아니다**

앞서 살펴본 Managed Training 서비스는 환경 구성과 운영, 장애 대응으로 학습 환경 준비, 장애 대응 등의 상황으로 낭비되는 GPU 시간을 줄여줄 수 있습니다. 하지만 학습을 효율적으로 운영하더라도 필요한 시점에 충분한 GPU를 확보하지 못하면 해당 학습 Job은 결국 대기하여야 합니다. 그렇다고 바로 학습용 GPU를 추가로 확보해야 하는 것은 아닙니다. 시야를 학습 클러스터 밖으로 넓히면, 이미 사용자 조직이 보유하고 있으면서도 특정 시간대에는 충분히 활용되지 않는 GPU가 존재할 수 있기 때문입니다. 대표적인 것이 서빙용 GPU입니다.

LLM 서빙 인프라는 평균 요청 트래픽이 아니라 피크 요청 트래픽에도 서비스 품질을 유지할 수 있도록 준비해야 합니다. 반면 실제 요청량은 하루 종일 일정하지 않습니다. 업무 시간에는 요청이 몰리고 심야에는 크게 감소하는 것처럼 특정한 사용자 시간 패턴이 반복될 수 있습니다. 특히 업무용 서비스처럼 사용 시간대가 뚜렷한 경우에는 야간이나 휴일에 피크 대비 상당한 유휴 GPU가 발생합니다. 결국 한쪽에서는 학습 Job이 GPU를 기다리는데, 다른 한쪽에서는 피크 타임을 위해 확보한 GPU가 "항상" 사용되지 않는 자원 불균형이 생길 수 있습니다.

- **최대 요청량을 위해 확보한 서빙 GPU, 요청량이 낮은 시간에는 쓰임이 없을까?**

이러한 서빙 시간대별 생기는 여유 GPU 용량은 실제 대규모 LLM 운영 사례에서도 확인할 수 있습니다. DeepSeek이 2025년 2월 공개한 V3/R1 인퍼런스 시스템 운영 자료에 따르면, 서빙에 사용된 GPU 노드는 피크 시간에 278개까지 사용된 반면, 하루 평균 사용량은 226.75개였습니다. 각 노드에는 H800 GPU가 8장 탑재되어 있었으며, DeepSeek은 요청량이 낮은 심야 시간에는 일부 자원을 연구 및 학습 Job이 사용할

수 있게 전환하였다고 설명하였습니다.[2] 피크 규모인 278개 노드를 하루 종일 유지한다고 단순 가정하면, 평균 사용량과는 차이가 해당 용량의 약 18%에 미칩니다. GPU 시간으로 환산하면 하루 약 4.4시간에 해당하는 규모입니다. 물론 이 차이가 모두 실제 유휴 GPU였다는 의미는 아닙니다. 중요한 것은 피크를 위해 확보한 서빙 용량과 실제 평균적으로 필요한 서빙 용량 사이에 학습 등 다른 작업에 활용할 수 있는 시간대별 여지가 존재한다는 점입니다. 수천 장 규모의 GPU 클러스터에서는 이러한 실 서빙 사용량의 차이가 상당한 양의 유휴 GPU 자원이 됩니다. 그렇다면 다음 같은 질문이 자연스럽게 생깁니다.

"서빙에 필요하지 않은 동안에는 GPU를 학습에 빌려주고, 다시 요청량이 증가하는 시간에는 돌려받을 수 없을까?"

- **해법: 유휴 GPU를 빌리고, 서빙이 필요할 때 돌려받는다**

이러한 Mixed Workload 환경의 핵심 동작이 Loaning & Reclaim, 즉 GPU의 대여와 회수입니다.

동작은 크게 세 단계로 볼 수 있습니다: 첫째, 플랫폼은 서빙 요청량과 GPU 사용량을 관찰 및 감지합니다. 과거의 시간대별 요청 패턴뿐 아니라 실시간 트래픽과 GPU 메트릭을 함께 고려해, 일정 시간 동안 학습에 "대여(Loan)"해도 서비스 품질에 영향을 주지 않는 적절한 자원량을 찾는 것입니다. 둘째, 이렇게 판단된 GPU들을 학습 Job에 대여합니다. 학습 Job 입장에서는 별도의 전용 GPU가 확보될 때까지 기다리는 대신, 현재 사용 가능한 서빙 GPU를 이용해 먼저 학습을 진행할 수 있습니다.

셋째, 트래픽이 다시 증가하면 학습보다 서빙이 우선시되어야 하기 때문에 플랫폼은 요청량이 실제 피크에 도달하기 전에 GPU를 학습 Job에서 "회수(Reclaim)"하여 다시 서빙에 투입합니다. 회수가 늦어지면 서빙 서비스 품질에 영향을 줄 수 있기 때문에, 핵심은 단순히 회수하여 GPU를 되찾아주는 것이 아닙니다. 앞으로 서빙에 필요한 용량까지 고려해 언제 빌려주고 언제 돌려받을지를 결정하는 것이 Loaning & Reclaim의 핵심입니다.

- **대여 GPU에서 실행되는 학습은 중단한다, Spot Job**

Loaning & Reclaim을 적용하면 학습 Job의 성격도 달라집니다. 학습 Job 입장에서 앞서 대여받은 GPU는 학습 전용 자원이 아닙니다. 서빙 트래픽이 증가하면 언제든 회수될 수 있습니다. 따라서 이 자원에서 실행되는 학습은 중간에 자원이 빼앗길 수 있다는 것을 전제로 실행되는 Job입니다. 이러한 대여한 GPU에서 도는 학습은 회수를 전제로 설계된 작업, 이른바 Spot Job이어야 합니다. 학습 Job은 기다리면 순서대로 실행이 되지만 Spot Job은 서빙 유휴 GPU가 발생해야 구동되는 실행이 보장되지 않는 작업을 의미합니다. 즉, Spot Job은 회수 가능한 GPU를 활용하는 대신 실행 도중 자원이 줄어들거나 Job이 중단될 수 있음을 전제로 합니다.

반복 실험, 완료 시점이 비교적 유연한 Batch 작업은 이러한 자원을 활용하기 좋은 후보입니다. 반면 특정 시점까지 반드시 완료되어야 하거나 중단 비용이 매우 큰 학습 Job이라면 안정적으로 보장된 전용 자원에 배치하는 편이 적합합니다. 여러 사용자가 유휴 GPU를 함께 요청한다면 공정성(Fairness)도 고려해야 합니다. 먼저 요청한 Job이 계속 유휴 자원을 독점하지 않도록 사용자 할당량(Quota), Job 우선순위(Priority), 대기 시간과 운영 정책 등을 함께 고려하여 대여 대상을 결정해야 합니다.

- **Spot Job의 연산 유실을 최소화하려면?**

Loaning & Reclaim은 새로운 GPU를 추가하지 않고도 기존 클러스터의 유휴 자원을 학습에 활용할 수 있게 합니다. 하지만 GPU를 회수할 수 있다는 조건은 새로운 문제점을 만듭니다. 예를 들어 3시간 동안 학습한 Job에서 GPU가 회수된다고 가정해 봅시다. Job을 다시 실행 할 수 있다고 해도 3시간 동안 생성된 학습 결과를 저장하지 않는다면 3시간 규모의 학습에 사용된 GPU 연산 값이 그대로 사라집니다. 그렇다고 학습 상태가 지나치게 자주 저장하면 모델이 거대해질수록 중간 결괏값인 체크포인트를 저장하는 동안 수 분에서 길게는 수십 분의 대기 시간이 발생합니다.

첫 번째 문제는 "GPU가 언제 회수되더라도 지금까지 학습된 모델 파라미터를 어떻게 효율적으로 지킬 것인가?"입니다. 하지만 또 하나의 문제가 더 있습니다. 일부 GPU만 회수되는 상황에서 매번 학습이 대여한 전체 GPU를 클러스터에 돌려줘야 할 까요? 회수하고 남은 GPU로 계속 학습은 불가할까요? 가능하다면 더욱 GPU 사용률을 높일 수 있습니다.

두 번째 문제는 "학습에 사용할 수 있는 GPU 수가 변경되더라도 학습을 계속 이어갈 수 있을까?"입니다. 이어지는 다음 내용으로는 첫 번째 문제, 다시 말해 GPU 회수 시 학습 진행 분을 최대한으로 유지하면서 이때 체크포인트 저장으로 인한 GPU 대기 시간을 줄이는 방법을 살펴보겠습니다.



[그림4] 시간대별 Serving GPU와 Training GPU의 대여·회수 (출처: 삼성SDS)

빌린 GPU를 돌려줘야 할 때, 학습 진행 분은 어떻게 지킬까?

- **체크포인트는 학습 진행 분을 지켜주지만, 공짜가 아니다**

GPU 회수로 학습 Job이 중단되더라도 이전 상태에서 다시 시작하려면 체크포인트가 필요합니다. 체크포인트는 일정 시점의 학습 상태를 저장해 두었다가 장애나 추가 학습을 위해 이후 해당 기점에서부터 학습을 이어갈 수 있도록 하는 모델 정보 파일입니다. 여기서 저장해야 하는 것은 모델 파라미터만이 아닙니다. 학습을 가능한 한 동일한 상태에서 이어가려면 옵티마이저 스테이트(Optimizer State)를 비롯한 학습 상태와 진행 정보도 함께 저장해야 합니다. 특히 Adam 계열 Optimizer는 파라미터마다 모멘텀과 분산 추정치 등의 추가적 정보를 유지하기 때문에, 전체 체크포인트의 크기는 모델 파라미터 자체보다 훨씬 커질 수 있습니다. 수십B 규모의 모델에서는 한 번의 체크포인트가 수백 GB의 크기에 이를 수 있습니다. 이러한 용량은 저장 자체가 상당한 I/O 작업이 됩니다.

- **문제는 '얼마나 자주 저장할 것인가'이다**

GPU 회수로 인한 학습 중단에 대비하려면 체크포인트를 자주 남기는 것이 유리합니다. 예를 들어 30분마다 체크포인트를 저장한다면 갑작스럽게 GPU가 회수되더라도 최대 약 30분 동안의 학습만 다시 수행하면 됩니다. 반대로 몇 시간에 한 번만 저장한다면 한 번의 회수로 몇 시간의 GPU 연산을 다시 해야 할 수도 있습니다. 그렇다고 저장주기를 무조건 짧게 만들 수도 없습니다. 전통적인 동기식(Synchronous) 체크포인팅에서는 일관된 학습 상태를 저장하기 위해 학습을 멈추고 체크포인트 쓰기가 완료되기를 기다립니다. 앞서 말했듯 수

백 GB 규모의 데이터를 저장하는 동안 학습에 참여하는 GPU 역시 다음 학습 단계로 넘어가지 못한다면, 체크포인트를 자주 남길수록 GPU의 대기 시간이 늘어납니다. 즉, 두 비용이 서로 충돌합니다.

체크포인트를 자주 저장하면 학습 중단 시 잃는 학습량은 줄지만, 저장 때문에 GPU가 자주 실제 연산을 실행하지 못하고 기다립니다. 반대로, 체크포인트를 드물게 저장하면 평소에는 GPU를 계속 계산에 사용할 수 있지만, 학습 중단이 발생했을 때 마지막 체크포인트로부터 종료 시점까지 재연산해야 하는 학습량이 커집니다. 장애가 간헐적으로 발생하는 전용 학습 환경에서도 고민해야 하는 Trade-off지만, 자원 회수가 주기적이고 자주 일어나는 Spot Job에서는 문제가 커집니다. 회수가 반복되는 환경에서 체크포인트 간격까지 길다면, 같은 구간의 학습을 계속 다시 연산해야 하는 상황이 벌어질 수 있습니다. 이는 GPU 연산의 낭비로 직결됩니다. 따라서 문제의 핵심은 "GPU 연산 낭비를 줄이기 위해 체크포인트를 자주 남기면서도, 체크포인트 저장 시 생기는 GPU가 연산하지 않는 시간을 어떻게 줄일 것인가?"입니다.



[그림5] 동기 방식과 Concurrent Checkpointing의 비교 (출처: 삼성SDS)

- **해법: GPU 연산 시간과 저장 입출력 시간을 최대한 겹치는 방법**

Concurrent Checkpointing은 이 Trade-off를 저장과 학습을 겹치는 방식으로 풉니다. 전통적인 방식에서는 학습 상태를 캡처한 뒤 최종 스토리지에 기록하는 과정이 끝날 때까지 학습이 기다릴 수 있습니다. 반면 Concurrent Checkpointing에서는 먼저 학습을 재개하는 데 필요한 모델 정보를 빠른 저장이 가능한 메모리 계층으로 저장합니다. 이후 상대적으로 느린 복사와 저장 작업을 다음 학습 Step과 겹쳐 수행합니다.

먼저 저장할 학습 상태 정보 데이터를 CPU 메모리나 로컬 저장장치와 같은 빠른 계층으로 넘깁니다. 학습은 가능한 한 빠르게 다음 Step을 재개하는 동시에, CPU 메모리로 내려졌던 데이터는 백그라운드에서 다른 노드의 메모리나 로컬 스토리지, 최종적으로는 원격 Object Storage와 같은 내구성 있는 계층으로 복제됩니다.

이 접근은 주요 클라우드 사업자들이 이미 서비스 수준으로 제공하고 있습니다. Microsoft Azure는 Nebula라는 비동기 체크포인트 기능으로 대형 모델의 체크포인트 저장 시간을 시간 단위에서 초 단위까지 줄일 수 있다고 소개합니다. 공개된 측정 예시에서는 20.6GB 규모 모델의 체크포인트 저장 시간을 96.9% 단축했다고 밝히고 있습니다.[3] AWS는 2025년 9월 SageMaker HyperPod에 Managed Tiered Checkpointing을 출시했습니다. 체크포인트를 먼저 인접 노드들의 CPU 메모리에 복사해 두고(Fast Tier), 사용자가 정한 주기로 내구성 있는 오브젝트 스토리지에 저장하는(Durable Tier) 이중 구조로, 수백 장에서 15,000장 이상의 GPU 클러스터에서 초 단위 저장을 검증했다고 밝혔습니다.[4] Google Cloud 역시 RAM에 먼저 쓰고 클러스터 내 복제와 오브젝트 스토리지 백업을 이어가는 Multi-tier Checkpointing을 제공합니다.[5] 연구 쪽에서는 Amazon이 2023년 발표한 Gemini가 체크포인트를 다른 머신의 메모리에 분산 배치해 장애 복구 성공률을 높이는 기법을 제시한 바 있습니다.[8] 체크포인트를 얼마나 빠르게, 얼마나 자주 남길 수 있는지가 학습 인프라 서비스의 비교 항목으로 자리 잡고 있습니다.

- **회수가 잦아질수록 Concurrent Checkpointing의 가치가 커진다**

이제 다시 Loaning & Reclaim 상황으로 돌아가 보겠습니다. 서빙에서 남는 GPU를 학습에 대여하면 GPU 활용률은 높아지지만, 이 GPU는 언제든 원래 용도인 서빙으로 돌아갈 수 있습니다. 따라서 Spot Job의 효율은 단순히 GPU를 얼마나 오래 빌릴 수 있느냐뿐 아니라, 회수될 때 얼마나 적은 학습량을 잃느냐에 달려 있습니다. Concurrent Checkpointing을 통해 학습 상태를 짧은 주기로 안전한 계층으로 계속 넘겨줄 수 있다면, GPU가 회수되더라도 다시 계산해야 하는 범위를 최근 체크포인트 이후의 짧은 구간으로 제한할 수 있습니다.

이 차이는 Loaning 정책에도 영향을 줍니다. 회수될 때마다 오랜 학습 결과물을 잃는다면 짧게 남는 GPU에는 아예 학습 Job을 배치하지 않는 편이 나을 수 있습니다. 반대로 회수 비용을 충분히 낮출 수 있다면 몇 시간, 혹은 그보다 짧게 열리는 GPU 여유 시간도 학습에 활용할 수 있습니다. 즉, Concurrent Checkpointing 기술은 회수 가능한 GPU를 실제 학습 자원으로 활용할 수 있는 범위를 넓혀주는 기술이 됩니다.

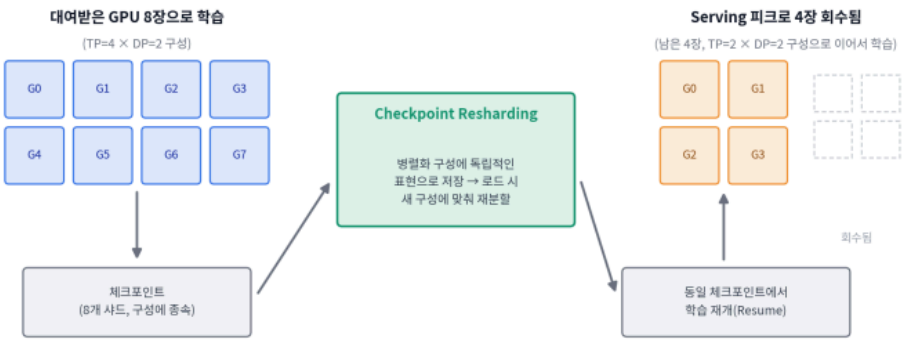
하지만 체크포인트를 빠르게 남겨 지금까지의 GPU 연산 결과를 지킬 수 있다고 해도 한 가지 문제가 남습니다. 32장의 GPU로 학습하던 중 8장이 회수됐다면, 반드시 나머지 24장까지 학습 Job에서 회수해야 할까요? 체크포인트가 있다고 해서 반드시 4장에서 바로 학습을 이어갈 수 있는 것은 아닙니다. 여기에는 분산학습 특유의 또 다른 제약이 있기 때문입니다. 다음으로는 이러한 제약 속에서도 학습을 계속 이어갈 수 있도록 만드는 체크포인트 리샤딩(Resharding) 방법에 대해 살펴보겠습니다.

GPU 수가 바뀌어도 학습을 이어갈 수 있을까?

- **체크포인트에도 저장 당시의 GPU 배치가 남아 있다**

분산학습에서는 모델과 Optimizer State를 여러 GPU가 나눠 가지고 있기 때문에, 체크포인트 역시 여러 조각의 의미인 샤드(Shard)로 분산 저장됩니다. 이 샤드의 구성은 당시의 GPU 수와 TP·PP·DP·ZeRO 같은 병렬화 방식에 영향을 받습니다. 예를 들어 8장의 GPU에서 학습한 상태를 저장했는데, 회수 이후 4장만 사용할 수 있다면 기존 상태를 그대로 불러올 수 없는 경우가 있습니다. 기존 8장에 나뉘어 있던 학습 상태를 새로운 4장 구성에 맞게 다시 배치해야 하기 때문입니다. 대여·회수가 반복되는 환경에서 매번 원래 GPU 수가 다시 모일 때까지 기다려야 한다면, 유휴 GPU를 활용하는 효과도 크게 줄어듭니다. 따라서 필요한 것은 다음과 같습니다:

"저장 당시와 GPU 구성이 달라도 현재 확보된 자원에 맞춰 학습을 재개할 수 있어야 합니다."



[그림6] Checkpoint Resharding: GPU 8장의 체크포인트로 4장에서 학습 재개 (출처: 삼성SDS)

- **해법: 구성에 독립적인 체크포인트**

Checkpoint Resharding은 체크포인트를 병렬화 구성에 독립적인 표현으로 저장해 두고, 로드하는 시점에 새 GPU 구성에 맞게 재분할(Reshard)하는 기술입니다. ByteDance가 NSDI '25에서 발표한 ByteCheckpoint는 병렬화에 독립적인(Parallelism-agnostic) 체크포인트 표현과 공유 메타데이터를 도입해, 로드 시 새 병렬 구조에 필요한 조각만 조합해 읽는 자동 Resharding을 구현했습니다. 비동기 I/O 파이프라인과 워크로드 분배 최적화를 더해, 기존 분산 체크포인트 대비 체크포인트로 인한 학습 정지를 크게 줄이고 저장과 로드 속도를 여러 배 높였다고 보고합니다.[6] Microsoft의 Universal Checkpointing도 체크포인트를 원자 단위(Atom Checkpoint)로 저장해 병렬화 전략과 하드웨어 구성에 종속되지 않는 구조를 제안했고, 병렬 구성 간 재구성 시간을 기존 대비 크게 단축했다고 밝혔습니다.[7] Resharding이 더해지면 대여·회수의 유연성이 크게 넓어집니다. 유휴 GPU가 몇 장이든, 그 수에 맞춰 학습을 이어갈 수 있기 때문입니다. 회수로 자원이 줄면 줄어든 구성으로 계속 학습하고, 심야에 자원이 다시 풍부해지면 큰 구성으로 확장해 학습 속도를 높이는 탄력적인 운영이 가능해집니다.

앞서 살펴본 Concurrent Checkpointing과 연결하면 역할은 더욱 명확해집니다. Concurrent Checkpointing은 GPU가 회수될 때 "얼마나 적은 학습 진행 분을 잃고 멈출 것인가"를 해결합니다. Checkpoint Resharding은 그 이후 "현재 남아 있거나 새로 확보된 GPU 구성에서 어떻게 다시 시작할 것인가"를 해결합니다. 두 기술이 함께 적용되면 유휴 GPU를 학습에 빌려주는 운영의 부담은 크게 낮아집니다. 서빙이 GPU를 필요로 하면 빠르게 학습 상태를 남기고 자원을 반환하고, 이후 GPU가 다시 확보되면 그때 사용할 수 있는 자원 규모에 맞춰 학습을 재개할 수 있기 때문입니다. 결국 Loaning & Reclaim이 남는 GPU를 학습에 사용할 기회를 만든다면, Concurrent Checkpointing과 Checkpoint Resharding은 그 GPU가 언제 얼마나 회수될지 모르는 환경에서도 학습을 지속할 수 있게 만드는 기반입니다.

이 기술들을 실제 GPU 운영에 어떻게 적용할까?

지금까지 살펴본 네 가지 유형은 서로 다른 문제처럼 보이지만 하나의 질문으로 귀결됩니다.

"비싼 GPU를 확보한 뒤, 그 시간을 얼마나 효율적으로 실제 연산에 사용할 수 있는가?"

이 기술들을 조합하면 하루의 운영 사이클이 그려집니다. 낮 동안 GPU Pool은 서빙에 전념합니다. 저녁이 되어 요청량 감소가 감지되면 유휴 GPU가 대기 중이던 Spot Job에 대여되고, 학습은 Concurrent Checkpointing으로 진행 상태를 하위 계층에 계속 남기며 밤새 진행됩니다. 아침 트래픽이 올라오기 전에 회수가 시작되면 학습 작업은 최신 체크포인트를 남기고 자리를 비켜주거나, Resharding을 통해 줄어든 구성으로 학습을 이어갑니다. 보유한 GPU Pool 전체가 시간대와 무관하게 일하는 구조입니다.

Samsung Cloud Platform의 Simple AI Training 서비스는 이 구조를 실제 운영 환경에 적용해 나가고 있습니다. Managed Training 서비스로 환경 구축의 부담을 덜고, 서빙 유휴 시간대의 GPU를 학습에 활용하는 Mixed Workload와 Concurrent Checkpointing을 결합해, 고객이 별도의 학습 전용 클러스터를 확보하지 않고도 학습 수요에 대응할 수 있도록 지원하는 방향입니다.

도입을 검토한다면 다음 항목을 먼저 확인해 보시기 바랍니다.

- 보유한 학습 워크로드를 중단 허용 여부에 따라 분류했는가? (중단을 허용하는 작업이 Spot Job의 후보입니다.)

- 체크포인트 저장이 학습 처리량에 주는 오버헤드와, 중단 후 재개까지 걸리는 시간을 측정했는가?
- 서빙 트래픽의 시간대별 프로파일을 확보해 대여 가능한 유휴 시간의 규모를 파악했는가?
- 학습 데이터와 체크포인트가 오가는 스토리지 경로의 대역폭이 비동기 복제를 감당하는가?
- 유휴 자원을 어느 팀의 어떤 작업에 먼저 배분할지, 우선순위와 공정성 정책을 정했는가?
- GPU 구성 변경 후에도 학습 결과가 재현되는지 검증하는 절차를 갖췄는가?

마치며

분산학습의 진입 장벽은 인프라·분산 소프트웨어·운영의 세 계층에 걸쳐 있고, Managed Training 서비스는 이 계층들을 대신 맡아 학습 시작까지의 준비 기간을 줄여줍니다. GPU 확보의 부담은 이미 보유한 서빙 GPU의 유휴 시간에서 답을 찾을 수 있으며, 이를 실현하는 기술이 대여·회수(Loaning & Reclaim)와 Concurrent Checkpointing, Checkpoint Resharding입니다. 다음 글에서는 이 구조를 받치는 나머지 축인 Multi-Clustering과 GPU Failover에 대해 실제 구현 관점에서 다룰 예정입니다.

References

[1] Meta AI. (2024). The Llama 3 Herd of Models. arXiv:2407.21783 (16,384 H100, 54일, 중단 419건)

[2] DeepSeek. (2025). DeepSeek-V3/R1 Inference System Overview. https://github.com/deepseek-ai/open-infra-index/blob/main/202502OpenSourceWeek/day_6_one_more_thing_deepseekV3R1_inference_system_overview.md

[3] Microsoft. Optimize Checkpoint Performance for Large Models (Nebula). https://learn.microsoft.com/en-us/azure/machine-learning/reference-checkpoint-performance-for-large-models

[4] AWS. (2025). Managed Tiered Checkpointing, Amazon SageMaker HyperPod. https://docs.aws.amazon.com/sagemaker/latest/dg/managed-tier-checkpointing.html

[5] Google Cloud. (2025). Using Multi-tier Checkpointing for Large AI Training Jobs. https://cloud.google.com/blog/products/ai-machine-learning/using-multi-tier-checkpointing-for-large-ai-training-jobs

[6] Wan, B., et al. (2025). ByteCheckpoint: A Unified Checkpointing System for Large Foundation Model Development. USENIX NSDI '25. arXiv:2407.20143

[7] Lian, X., et al. (2024). Universal Checkpointing: Efficient and Flexible Checkpointing for Large Scale Distributed Training. arXiv:2406.18820

[8] Wang, Z., et al. (2023). Gemini: Fast Failure Recovery in Distributed Training with In-Memory Checkpoints. https://www.amazon.science/publications/gemini-fast-failure-recovery-in-distributed-training-with-in-memory-checkpoints

▶ 이 아티클은 AI를 활용하여 제작되었습니다.

- 톤앤 매너 보정 및 도식화 : Claude

FAQ

Q1. 분산학습은 언제부터 필요합니까?

모델과 학습 상태가 GPU 한 장의 메모리에 들어가지 않거나, 단일 GPU로는 학습 기간이 감당할 수 없이 길어지는 시점이 분기점입니다. 모델 파라미터에 더해 옵티마이저 상태까지 저장해야 하므로, 실제 메모리 요구량은 모델 크기의 몇 배에 이릅니다.

Q2. Spot Job에는 어떤 학습이 적합합니까?

파인튜닝, 실험 성격의 학습, 마감이 유연한 배치 작업처럼 중단을 견디는 워크로드가 적합합니다. 완료 시점이 엄격하게 정해진 학습은 회수 위험이 없는 전용 자원에 배치하는 편이 안전합니다. 체크포인트 저장 전략이 갖춰져 있을수록 Spot Job으로 운영할 수 있는 범위가 넓어집니다.

Q3. Concurrent Checkpointing은 학습 성능에 영향을 주지 않습니까?

학습이 멈추는 구간은 GPU 메모리 스냅숏 시간으로 한정되고, 이후의 복사와 복제는 학습 연산과 병렬로 진행됩니다. 저장 빈도를 높여도 전체 학습 시간에 주는 영향이 적지만, 스냅숏 오버헤드와 스토리지 대역폭은 환경마다 다르므로 도입 전 측정을 권장합니다.

Q4. GPU 수가 달라져도 학습 결과가 같습니까?

Checkpoint Resharding으로 학습 상태 자체는 정확히 이어받을 수 있습니다. 다만 GPU 수 변화에 따라 전체 배치 크기와 학습률 같은 하이퍼파라미터의 재조정이 필요할 수 있고, 구성 변경이 잦으면 학습 안정성 검증이 필요합니다. 재현성이 중요한 학습이라면 구성 변경 이력을 함께 관리하는 것이 좋습니다.

Q5. Mixed Workload를 도입하면 서빙 품질이 떨어지지 않습니까?

회수(Reclaim)가 서빙 품질 저하보다 먼저 일어나도록 설계하는 것이 이 기술의 전제입니다. 서빙 요청량이 피크에 도달하기 전에 학습에서 GPU를 회수하며, 회수 판단은 시간대별 패턴과 실시간 메트릭을 함께 반영해 보수적으로 동작합니다. 서빙 SLA를 지키는 범위 안에서만 대여가 이뤄집니다.